



Technische
Universität
Braunschweig

**Decision
Support**

Institut für Wirtschaftsinformatik



Data Driven Decision Making

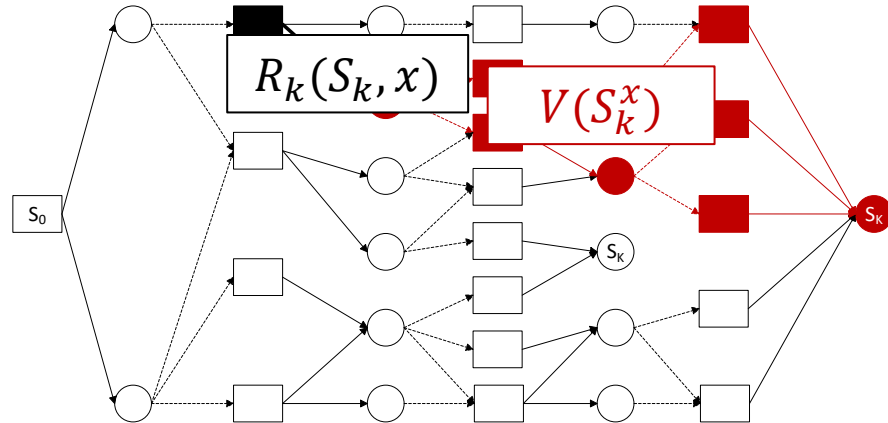
Lecture 8 – Value Function Approximation

Agenda

- Recap
- Motivational Experiment
- Value Function Approximation
- Case Study: Customer Acceptances + Approximation Architectures (if time is left)

Value Function

- Expected future reward is called the **value** $V(S_k^x)$ of post-decision state S_k^x
- The value function maps a (post-decision) state to the expected future reward



Policy Classes (Compare Powell 2011)

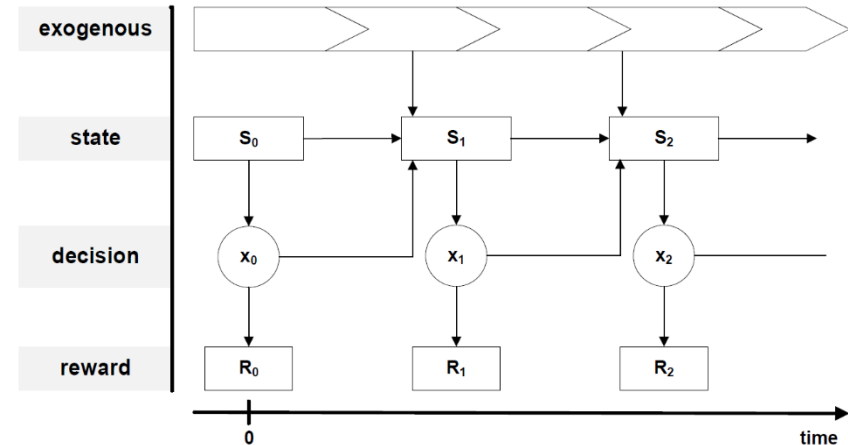
- Policy Function Approximation
- Rolling Horizon Re-Optimization
- Cost Function Approximation
- Look-ahead Models
- Value Function Approximation

Rule-Based

Optimization-Based

Policy Classes I

- Policy function approximation:
Transfer a rule-of-thumb into a policy
- Rolling-Horizon Re-Optimization:
Determine decision based on current information (myopic)
- Cost-Function Approximation:
Manipulate reward function or constraints to incentivize (/avoid) (in)flexible decisions
- None of the classes anticipates the future (information & decision model) explicitly



Policy Classes II: Lookahead

- Methods consider future changes via sampling of realizations
- Rollout algorithm:
 - Simulate paths of sampled realizations and decisions made by a base policy
 - Use simulated values with Bellman Equation
- Multiple Scenario Approach:
 - Sample sets of realizations (scenarios)
 - Solve scenarios individually
 - Determine most similar solution → decision
- Disadvantages: runtime, decision making in simulation

Excursus: Evaluating Policies

- For each instance setting:
 - Generate sufficiently large set of instance realizations
 - Different than training data in Lookaheads and VFAs
- Solve same realizations for each tested policy
- Statistical significance: Are the differences just noise?
 1. Calculate standard errors: standard deviation divided by square root of observations.
 2. If better value minus its standard error larger than worse value plus its standard error, results are significant.

Calculations

Mean value

Variance

Standard dev.

Standard error

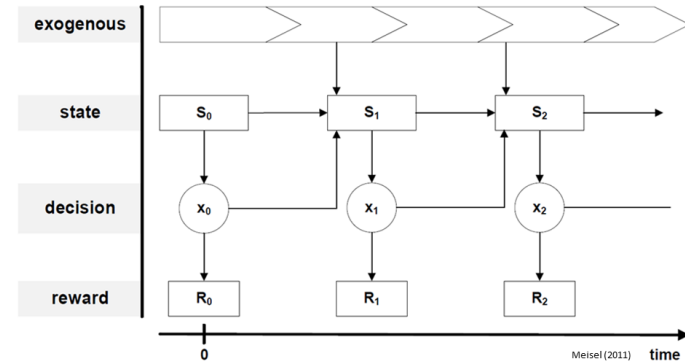
Significance

Agenda

- Recap
- **Motivational Experiment**
- Value Function Approximation
- Case Study: Customer Acceptances + Approximation Architectures

Motivational Experiment: Fair (dynamic knapsack problem)

- Fair with an unknown number of rides
- Time horizon (**60 minutes**)
- Money (expires at end of horizon) (**15€**)
- Random occurrences of rides
- Every ride costs money and generates fun (reward)
- Question: buy ride or not (no way back)
- Then, next (unknown) point of time with next ride
- Maximize the expected fun
- Value function: expected future fun when having a certain amount of money at a point of time



Dynamic knapsack problem

- Markov Decision Process:
- Decision point: Next item
- State: Point of time, remaining capacity (money), new item
- Decision: Take item (or not)
- Reward: Value of item (or 0)
- Post-decision state: Point of time, remaining capacity
- Stochastic information: realization of new item
- Transition: next point of time and item
- Final state: End of horizon
- Objective: Maximize expected reward

Experiment

Ride	Time	Reward (in fun)	Cost (in €)	Decision	Money	Total Fun
-	0	-	-	-	15	0

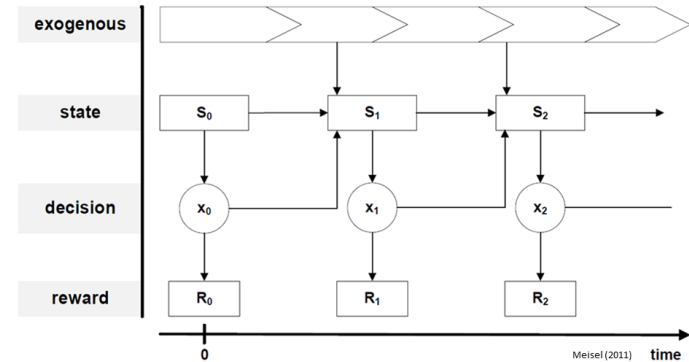
Experiment

Ride	Time	Reward (in fun)	Cost (in €)	Decision	Money	Total Fun
-	0	-	-	-	15	0
1.	3	10	5			
2.	16	11	5			
3.	18	8	3			
4.	21	7	4			
5.	23	10	7			
6.	50	9	3			
7.	57	8	1			

that's it

What could we learn?

- Information Model:
 - Expected number of rides per visit
 - Relation between costs and rewards
 - Changes over time
- Decision Making:
 - Approximate value of money at a point of time
 - Decision between buying (or not)
 - Value depends on our decision making
 - Repeated visits necessary (learning)



$$\max_{x \in X(s_k)} (R(s_k, x) + V(s_k^x))$$

What do we need to learn?

- Exogeneous information: Sampling from distribution
- Experience of interactions with exogeneous information: Simulation
- Decisions to generate interactions: Bellman Equation
- Memory to store and update experience: Value function
- We repeatedly simulate sampled information, decide with the Bellman Equation based on the value function and update the value function with the simulation's outcome

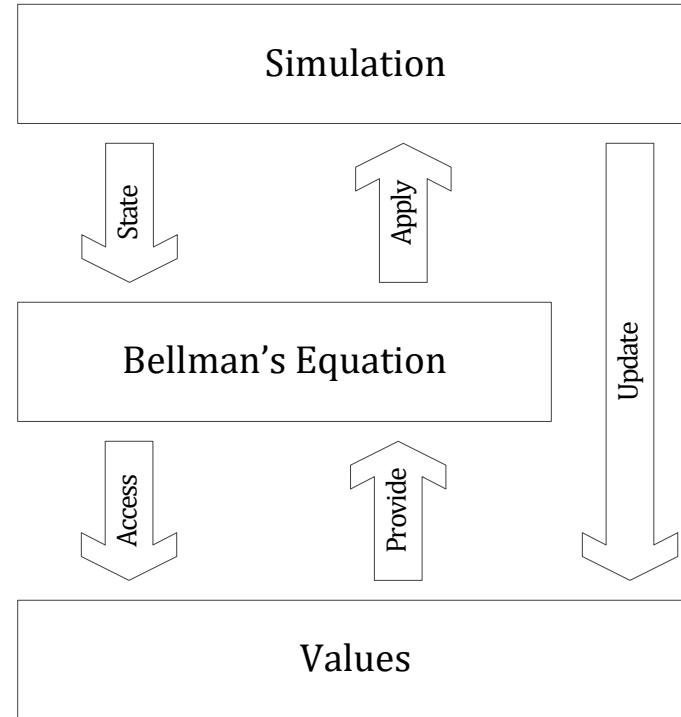
Agenda

- Recap
- Motivational Experiment
- **Value Function Approximation**
- Case Study: Customer Acceptances + Approximation Architectures

Value Function Approximation (Powell, 2011)

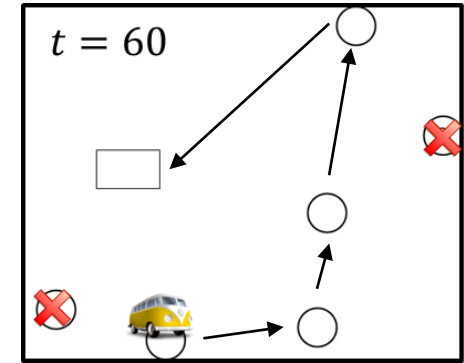
$$X_k^{\pi^*}(S_k) = \arg \max_{x \in X(S_k)} \{R_k(S_k, x) + V(S_k^x)\}$$

- Initial values $\hat{V}_0$
- Generate policy π_0^{VFA} w.r.t. $\hat{V}_0$
- Large number of simulations N
- Apply **anticipatory** policies π_i^{VFA} within run $i + 1$
- Update $\hat{V}_i$ after each simulation run
- Output: policy π_N^{VFA}
- Apply π_N^{VFA} in online execution
- Main challenge is storing the values



Storing the Approximate Value Function

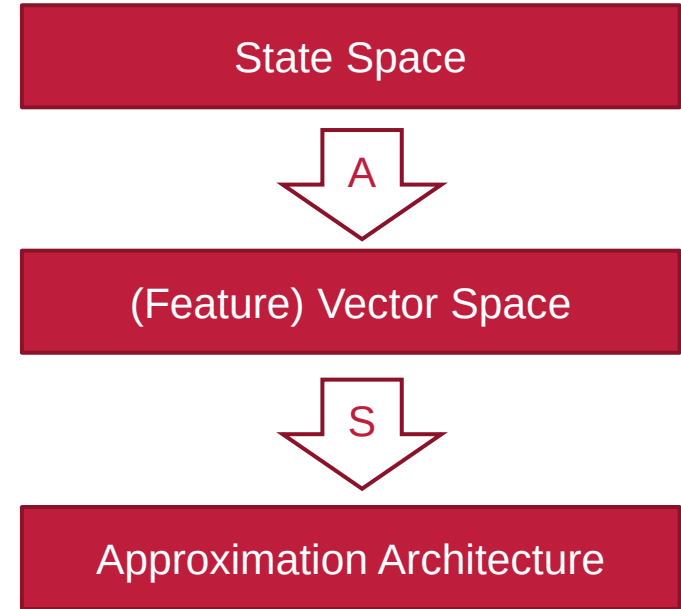
- Curses of Dimensionality
- For example, dynamic routing for courier services
- Post-decision state:
 - Point of time
 - Set of customers
 - Vehicle's position
 - Planned tour
- We never observe the same post-decision state twice.
- We cannot learn anything...



VFA: State Space Representation

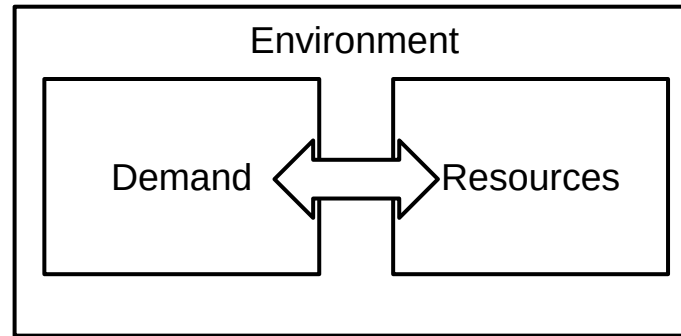
- Reduced representation of state space necessary
- Two steps:
 - Aggregation summarizes post-decision state to a vector of features
 - The value for each vector is stored using an approximation architecture

	Attribute 1	...	Variable	
Instance 1	$value_{11}$	$value_{1...}$	$value_1$	phase
...	$value_{...1}$	$value_{.....}$	$value_{...}$	training
Instance n	$value_{n1}$	$value_{n...}$	$value_n$	test data



Decision Making in Operations Research

- Fulfilment of demand with resources within an environment
- Demand: Products, service, delivery, transport, etc....
- Resources: Workforce, machines, delivery/service fleet, budget
- Environment: Street network, weather conditions, energy prices, etc.
- Objective: Maximize fulfilled demand vs. minimize used resources
- Decisions: Assignment of resources to demand.
- Environment often defines constraints and resource consumption.



Aggregation: Feature selection

- Features can represent environment, resources, and demand
- “Feature selection is an art” (Warren Powell)
- Environment
 - Point of time: later time in process, smaller value
 - Traffic level: higher traffic, smaller value (less revenue)
 - Energy prices (could be resource/demand):
 - Higher prices, smaller value (if we need to buy energy)
 - Higher prices, larger value (if we can sell energy)
 - Weather: Bad weather, small value (obstacles in delivery)

Feature selection: Resources

- Remaining vehicle capacity: more capacity, higher value
- Distribution of vehicles in the city: how to measure? Higher spread, higher value(?)
- Inventory levels: higher levels, higher value(?)
- Remaining advertising budget: higher budget, higher value(?)
- Number of workers in the system: more workers, higher value
- Percentage of skilled workers: higher percentage, higher value(?)
- Free server/production lines: more capacity, higher value
- Number of broken production lines: smaller number, higher value

Feature selection: Demand

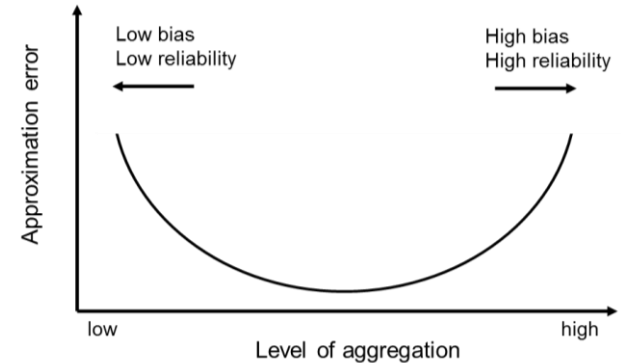
- Number of customers/jobs: higher number, smaller value
- Number of different job types in the system: higher number, smaller value
- Average spread in customer locations: higher spread, smaller value
- Customer satisfaction level: higher level, higher value
- Urgency: how to measure? Higher urgency, smaller value

Feature selection: Combination

- Average travel times between customers (Environment + Demand) high times, smaller value
- Time a production line is empty again (Resource + Demand): nearer times, higher value
- Average time between job deadlines and planned finish time (Resource + Demand): larger times, higher value

Aggregation: Bias vs. Reliability

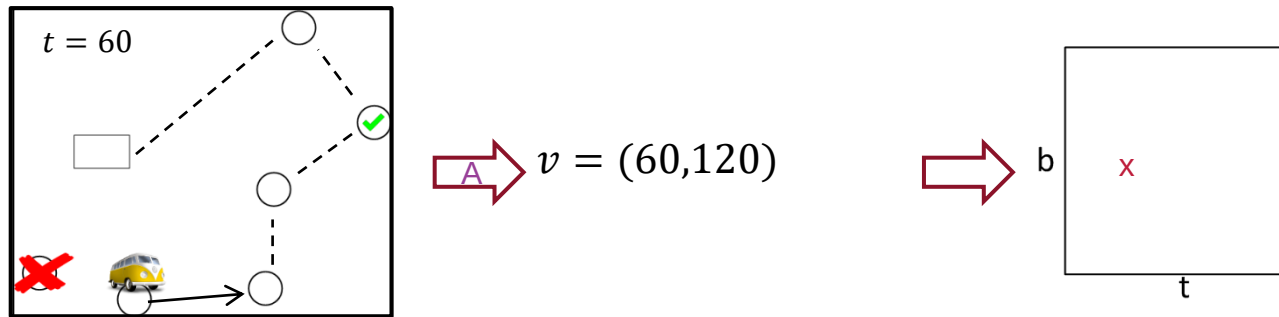
- Bias: deviation from aggregated to real value
- Reliability: enough feature observations
- Not enough vs. too many features
- Too general vs. too detailed features
- E.g., number of customers/jobs vs. individual customer locations / job specifications



Compare Hastie et al., 2016

Example: Courier Service

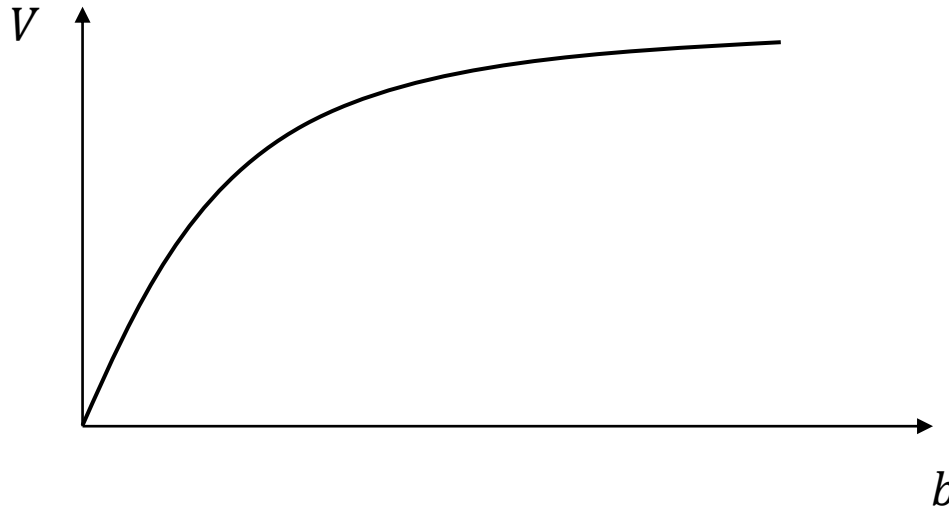
- Value significantly depends on point of time t and remaining time budget b (time limit – current arrival time at depot)
- Aggregate S_k^x to a 2-dimensional feature vector: $A(S_k^x) = v = (t, b)$



Ulmer, Mattfeld, Köster, Budgeting Time for Dynamic Vehicle Routing with Stochastic Requests, Transportation Science, 2018

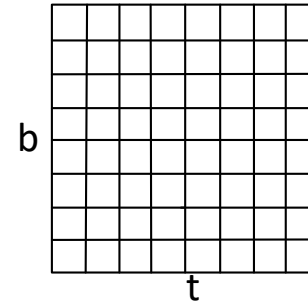
Value function for a fixed point of time (Assumption)

- Monotonic increase
- Saturation
- Compare with dynamic knapsack problem



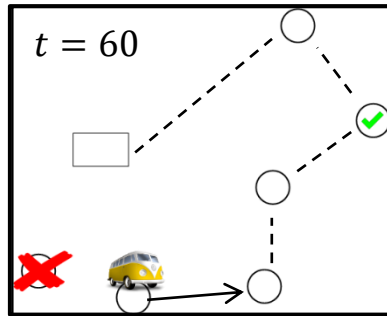
Approximation Architectures

- Input: Vector
- Output: Value
- Can be trained
- Examples:
 - Lookup tables
 - Parametric architectures (linear functions, etc.)
 - Neural nets
 - Decision trees
 - Etc.
- Combinations possible (see case study)

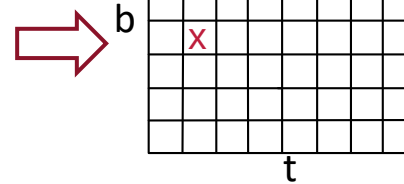


Courier Service: Lookup Table

- Aggregate S_k^x to a vector: $A(S_k^x) = v = (t, b)$
- Store values in lookup table with intervals
- Update entry values with running average



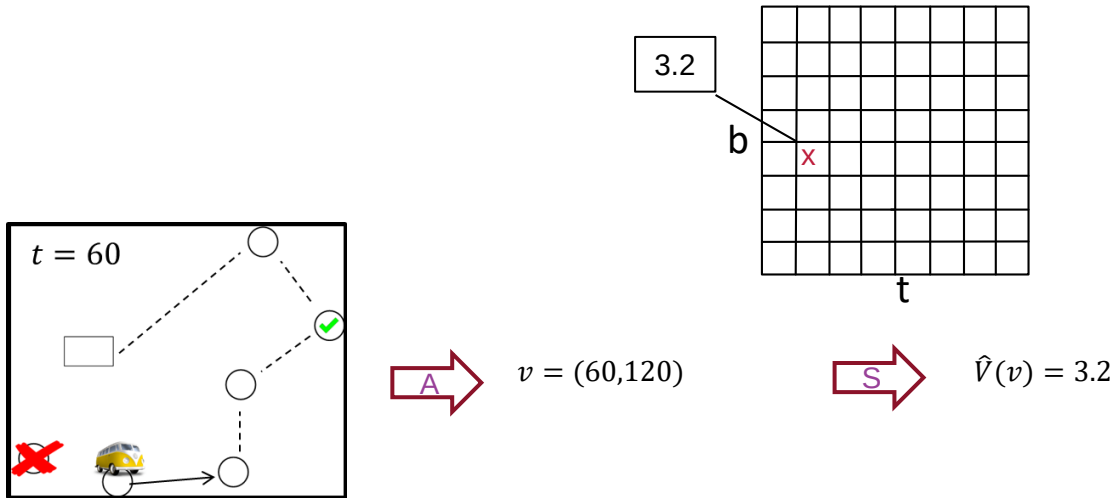
$\Rightarrow v = (60, 120)$



Courier Service: Decision Making

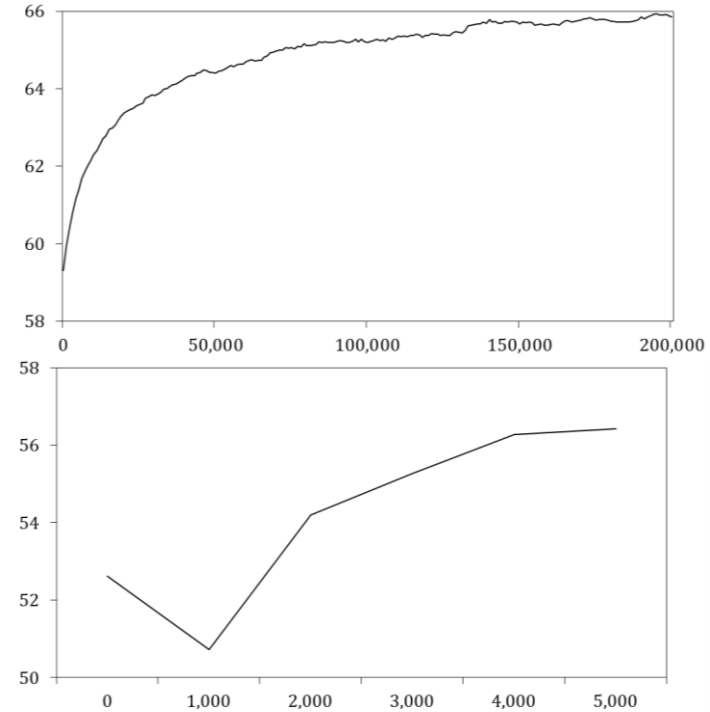
$$X_k^{\pi^*}(S_k) = \arg \max_{x \in X(S_k)} \{R_k(S_k, x) + V(S_k^x)\}$$

- Use current value for decision making via Bellman Equation
- E.g., current value of vector is 3.2



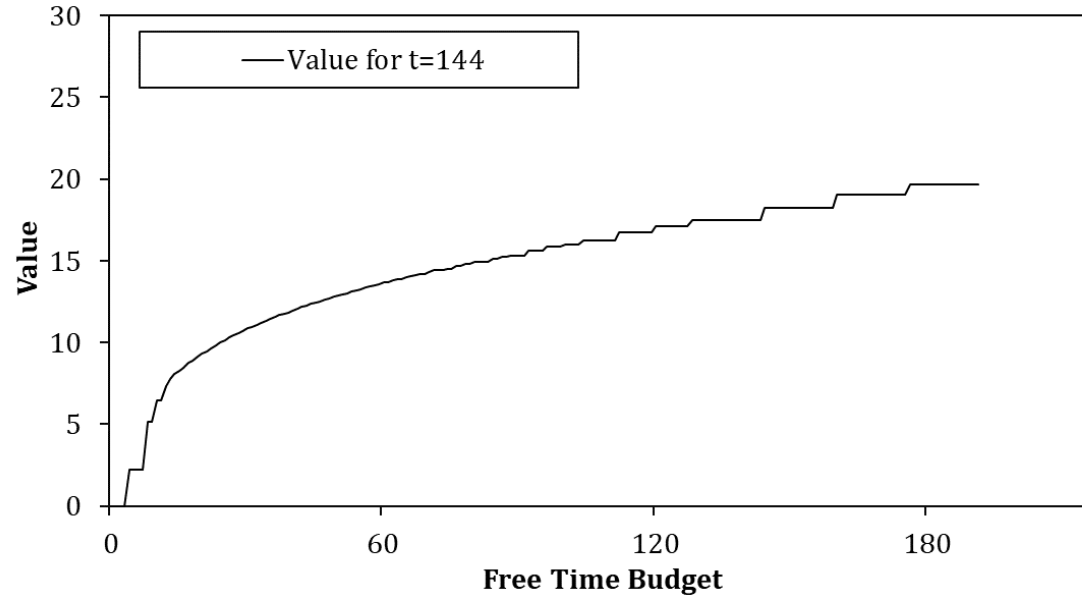
Courier Service: Learning Process

- Percentage of served customers
 - First 200,000 runs
 - Moving average over 10,000 runs
 - Fast increase
 - Saturation
-
- First 5,000 runs
 - Average over 1,000 runs
 - Initial decrease
 - Exploration (“trial and error”)



Value function for a fixed point of time

- Monotonic increase
- Saturation

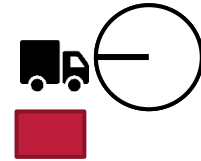


Agenda

- Recap
- Motivational Experiment
- Value Function Approximation
- **Case Study: Customer Acceptances + Approximation Architectures (if time is left)**

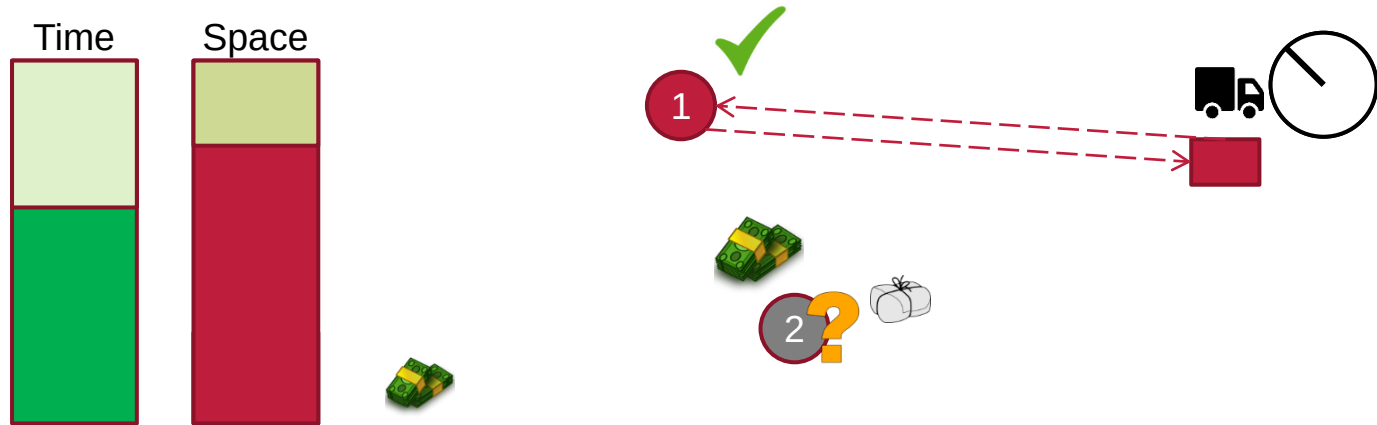
Dynamic Customer Acceptance Problem

- Capacitated vehicle
- Driver with working hours
- Customers request **next-day** delivery over the day
- Request comprises location, size, and revenue
- Decisions: Acceptance or rejection of request and routing



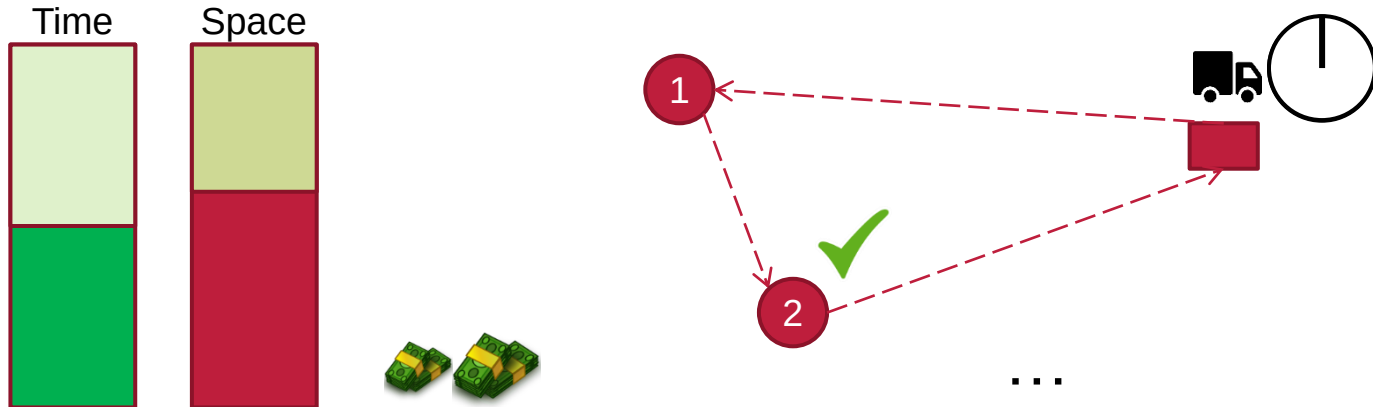
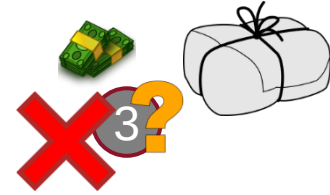
Dynamic Customer Acceptance Problem

- Capacitated vehicle
- Driver with working hours
- Customers request **next-day** delivery over the day
- Request comprises location, size, and revenue
- Decisions: Acceptance or rejection of request and routing



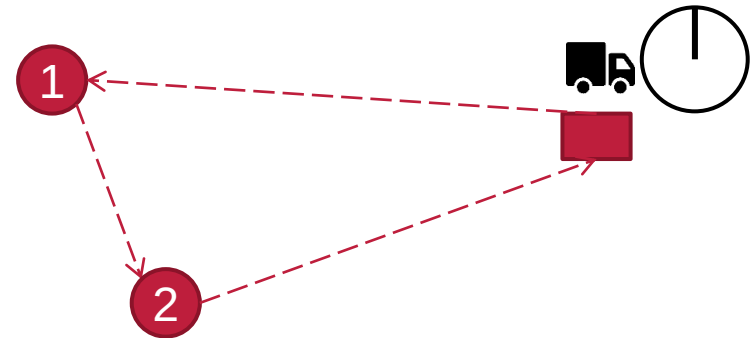
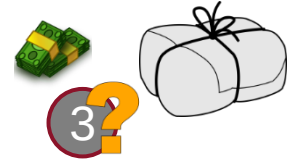
Dynamic Customer Acceptance Problem

- Capacitated vehicle
- Driver with working hours
- Customers request **next-day** delivery over the day
- Request comprises location, size, and revenue
- Decisions: Acceptance or rejection of request and routing



Dynamic Customer Acceptance Problem

- Markov Decision Process
 - Decision point: New request
 - State:
 - Point of time
 - Accepted customers and tour
 - New request
 - Decision:
 - Acceptance/Rejection
 - Integration in tour
 - Reward: Revenue (if accepted)
 - Stochastic information: New request
 - Transition: New time and state with new request
- Dynamic Knapsack Problem with Routing Component



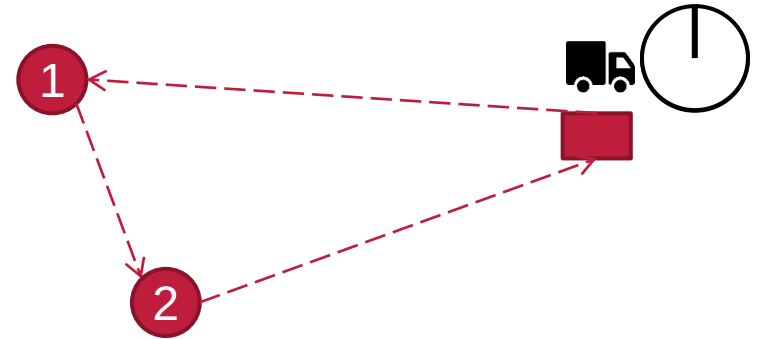
Challenges

- (Post-Decision) State space is too large
 - Point of time
 - Customer locations
 - Planned route

→ State space aggregation

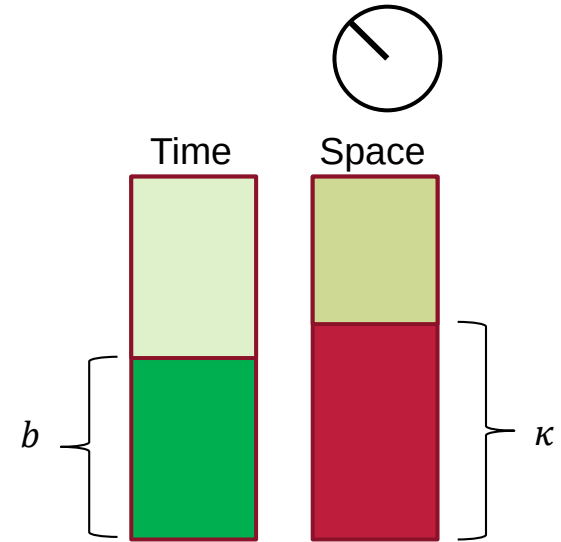
- How to model the value function?
 - Parametric (linear function)
 - Non-parametric (lookup table)

→ Meso-parametric combination



State Space Aggregation

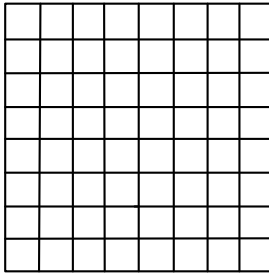
- Representation of a state by a set of parameters:
- Point of time t
- Free capacities
 - In Time b
 - In Space κ
- State is represented by vector (t, b, κ)
- Approximate $V(t, b, \kappa)$



How to model the value-function?

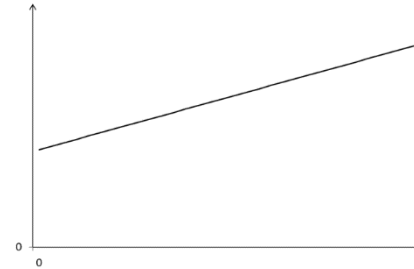
Non-Parametric

- Approximate individual values
- Partitioning of vector space
- Storage in a lookup table



Parametric

- Approximate parameters of a (linear) functional form
- Determine parameters via regression



How to model the value-function?

Non-Parametric

- Approximate individual values
- Partitioning of vector space
- Storage in a lookup table
- Advantage: local detail (less bias)
- Disadvantage: less reliable
- Works well for routing (Ulmer et al. 2018)

Parametric

- Approximate parameters of a functional form
- E.g., linear approximation
- Determine parameters via regression
- Advantage: global reliability
- Disadvantage: bias
- Works well for knapsack problems (Bertsimas and Demir 2002)

→ Combination of advantages

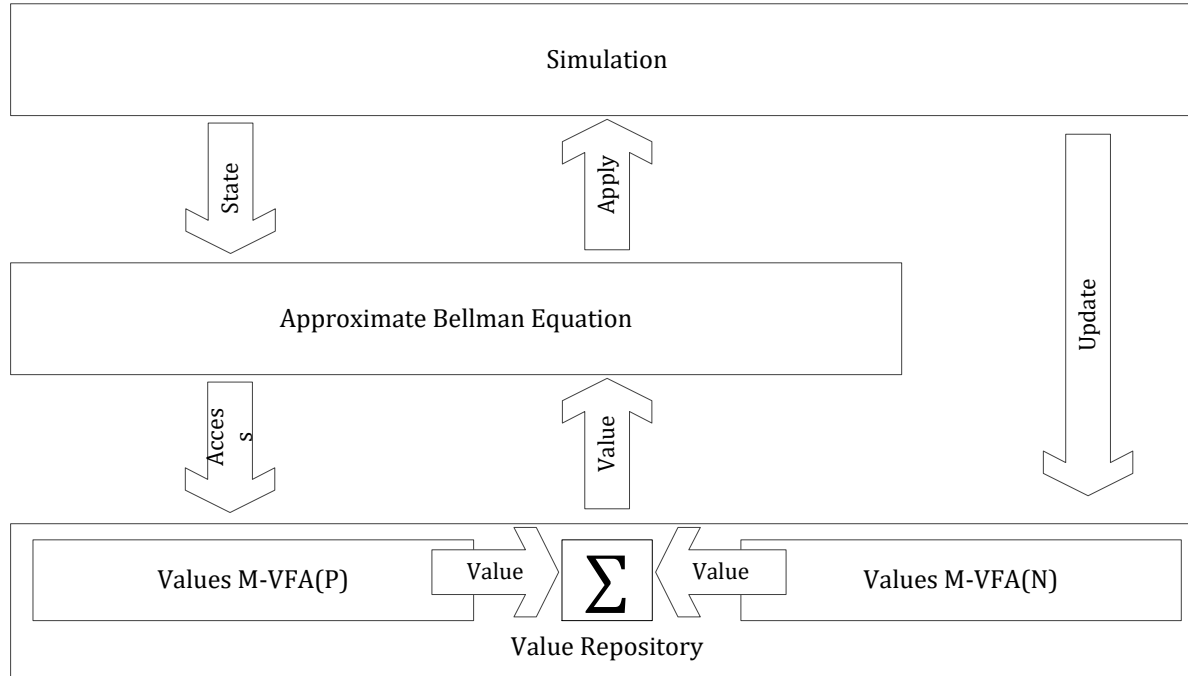
Meso-Parametric VFA

- Combination of Parametric (linear) and Non-Parametric (lookup table) VFA
- Approximated value function V^m

$$V^m(S_k^x) = \lambda V^p(S_k^x) + (1 - \lambda)V^n(S_k^x)$$

- Procedure:
 - Approximate both VFAs simultaneously: V^p, V^n
 - Determine value as weighted combination
- Tuning:
 - Determination of λ by enumeration: $\lambda = 0.0, 0.1, \dots, 1.0$
 - Special case: Lookup-table entry V^n may be empty (no observation yet)
 - In that case, only draw on V^p

Procedure



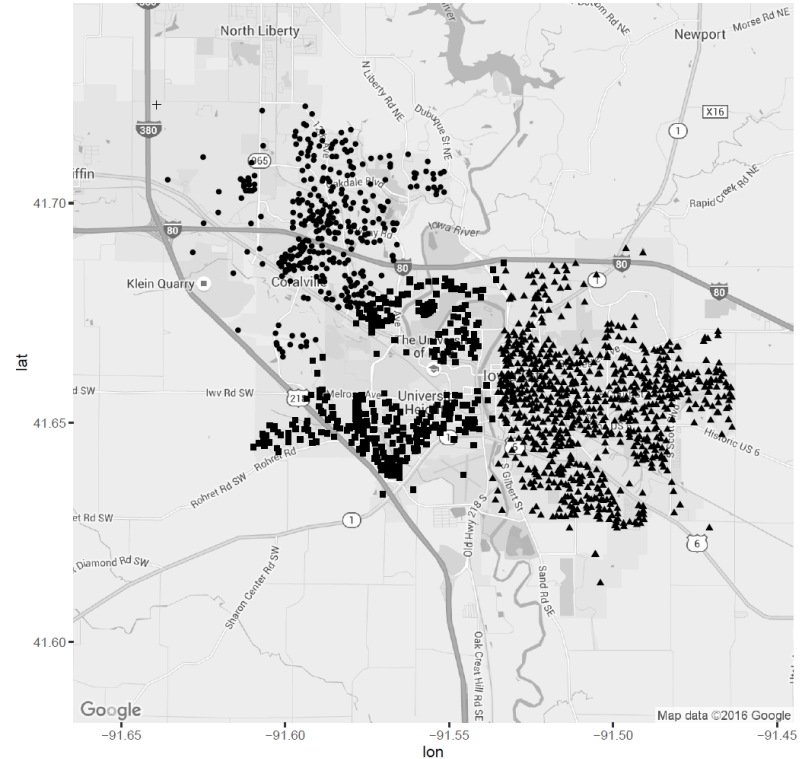
Computational Evaluation

- Time: 480 minutes
- Spatial capacity:
 - medium van
 - large van
 - small truck
 - medium truck
- Shift in bottleneck resource:
 - High capacity: routing problem (N-VFA beneficial)
 - Low capacity: knapsack problem (P-VFA beneficial)



Customer Data

- Number of requests: 50
- Customer locations: Iowa City
- Routing: Cheapest Insertion
(works well for Iowa City,
Ulmer and Thomas, 2019)

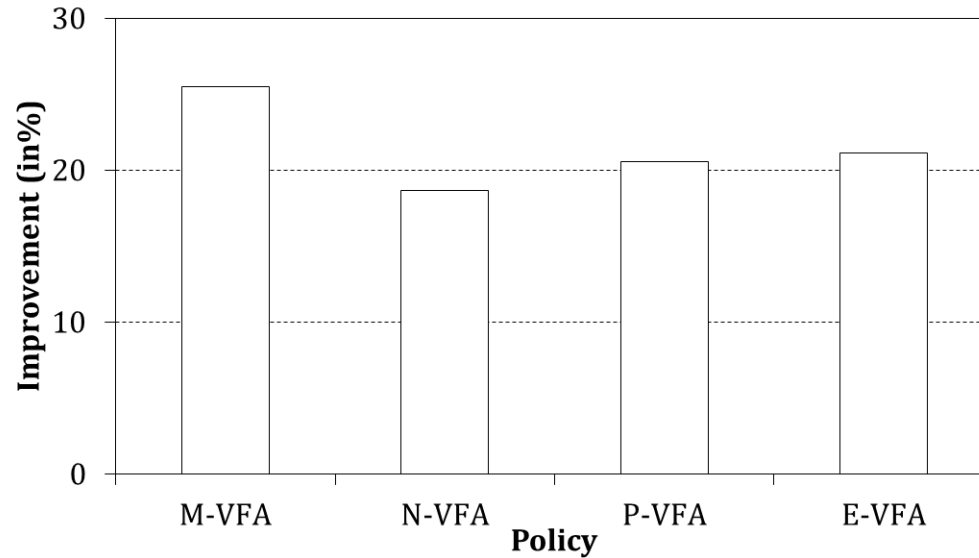


Policies

- VFAs:
 - M-VFA: Simultaneous approximation of M-VFA(N) and M-VFA(P)-components
 - N-VFA: Only Non-Parametric VFA
 - P-VFA: Only Parametric VFA
 - E-VFA: Ex post – combination (What is the value of simultaneous approximation?)
- Benchmark:
 - Rollout Algorithm
 - Simulate the rest-of-the-day from a current state online
 - Within simulations, accept every feasible customer (myopic base policy)

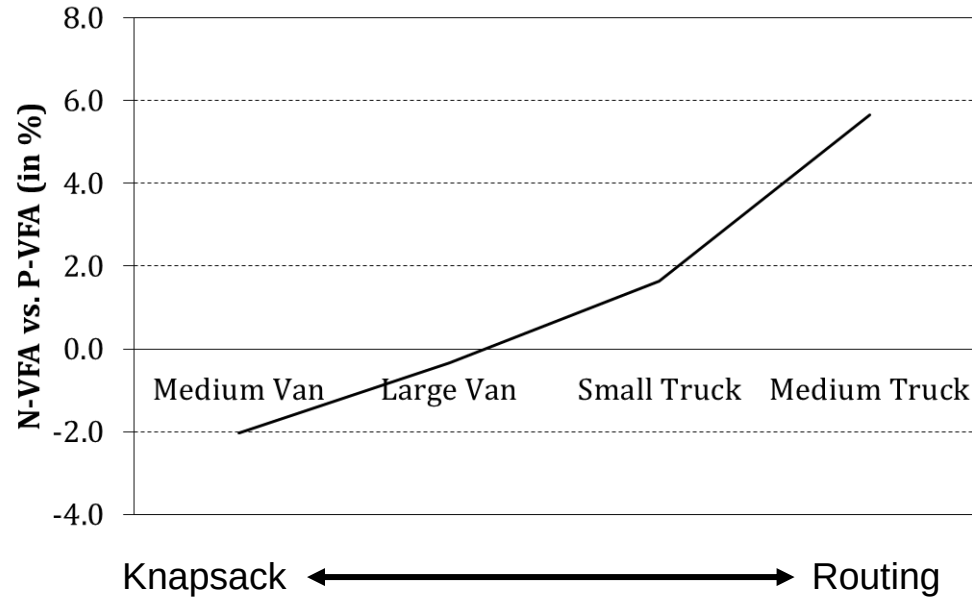
Results: Comparison to Rollout Algorithm

- All VFAs are substantially better
- Rollout algorithm with myopic base policy results in poor approximations.

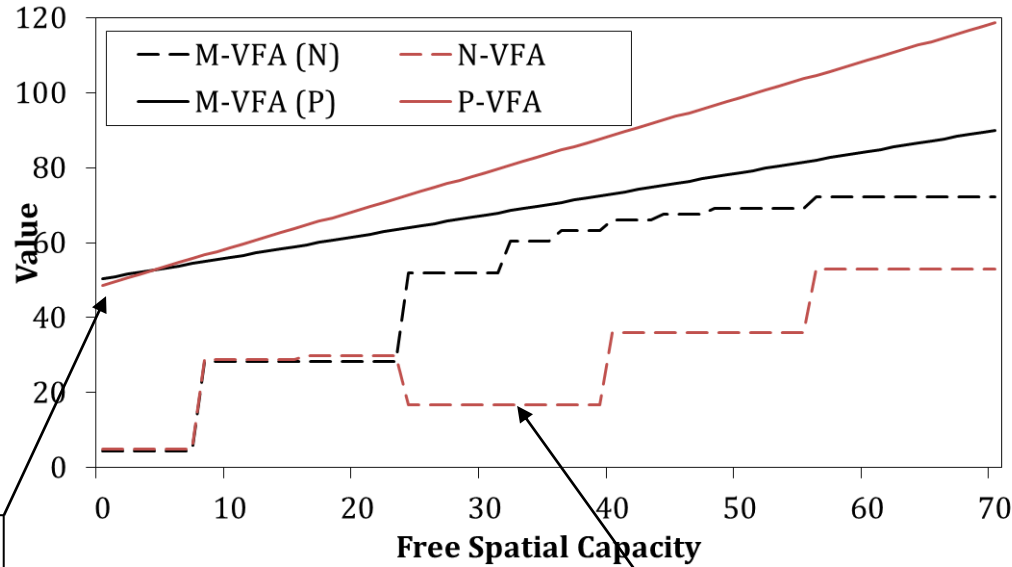


Knapsack vs. Routing Problem

- With larger vehicles capacity is no longer an issue: The knapsack component of the problem vanishes
- Instead, the routing component becomes more important and non-parametric VFA dominates



Approximated Value Function

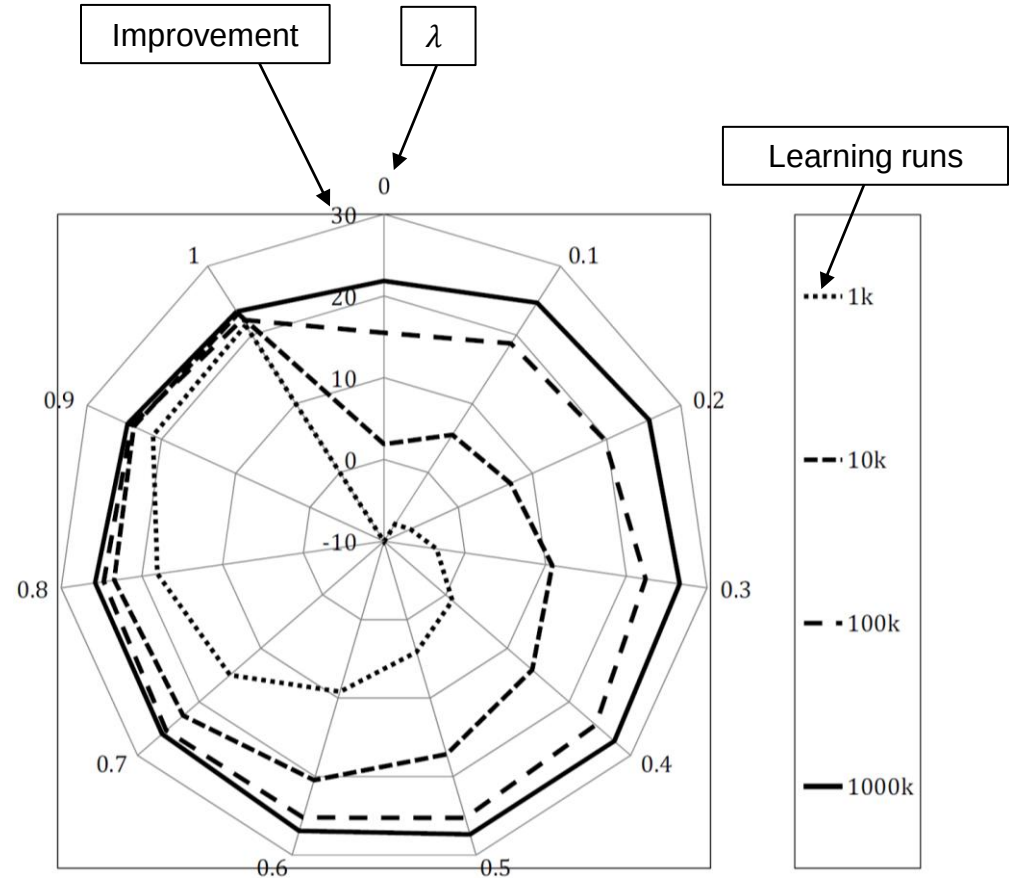


Zero capacity leads to value of zero (in theory)

Monotonicity in value function

Approximation Process

- Varying λ
- Comparison to rollout
- $\lambda = 0$: non-parametric
- $\lambda = 1$: parametric
- We see differences in approximation speed
- Individual approximations inferior



Summary of the Case Study

- Value function approximations can consider longer-term effects of decisions and anticipatory decision making (in contrast to a rollout algorithm)
- The approximation architecture should be selected with respect to the (assumed) value function shape/behavior:
 - Generality vs. Detail
 - Less reliability vs. Less bias

Summary

- Value function approximations can shift calculation effort to a learning phase
- Steps:
 - Aggregation to state features
 - Storage of feature vectors in approximation architecture
- Advantages:
 - Fast
 - Consideration of information and decision space
- Disadvantage:
 - Scalability
 - Missing details
 - Challenging to capture heterogeneous data
- To be done in lecture 10: combination of policy classes